Introduction à la cryptographie
Devoir Maison – Implémentation du cryptosystème de Pailler

A rendre pour le 7 Mai 2026
M1, Université de Lorraine, 2025/2026
Marine Minier, `marine.minier@loria.fr`

1 Introduction

Ce devoir est à réaliser par groupe de deux étudiant-e-s. Au plus, un seul trinôme sera accepté.

Le rendu se fera sur ARCHE et est en deux parties :

- Un rapport dactylographié au format PDF contenant les réponses aux questions posées dans le sujet. Il vous est demandé de rédiger correctement ces réponses, de détailler votre raisonnement et de justifier vos affirmations.
- Les programmes que vous avez développés. Le `main` de votre programme devra suivre le format de celui décrit en Annexe B. La sortie de votre programme devra être assemblée dans un unique fichier appelé `test.txt`.

L'ensemble des fonctions sera développé dans le langage de votre choix.

Les objectifs de ce DM sont les suivants :

- Comprendre le fonctionnement du cryptosystème de Pailler.
- Implémenter ce cryptosystème.
- Tester votre implémentation.

2 L'algorithme de chiffrement de Pailler

Le premier but de ce DM est d'implémenter le schéma de chiffrement de Pailler.

La sécurité de cet algorithme repose sur la difficulté de la factorisation: étant donnés p et q deux grands nombres premiers, connaissant p et q, il est facile de calculer $n = p \cdot q$ mais si on ne les connait pas, il est très difficile de trouver la factorisation de n.

Le cryptosystème de Paillier est un cryptosystème basé sur un algorithme asymétrique conçu par Pascal Paillier en 1999. Son principe repose sur des travaux de Okamoto et Uchiyama présentés en 1998.

2.1 Fonctionnement

Génération des clefs. `KeyGen()`.

- Choisir deux nombres premiers de grande taille, indépendants et aléatoires : p et q ;
- Calculer la clef publique $\mathsf{pk} \triangleq N = p \cdot q$ (un module RSA) et la clé privée $\mathsf{sk} \triangleq \varphi(N) = (p-1) \cdot (q-1)$.

Chiffrement. `Enc()`. Soit m un message à chiffrer avec $0 \leq m < N$. Soit r, un entier aléatoire tel que $0 < r < N$ (appelé l'aléa). Le chiffré est alors :

$$c = (1+N)^m \cdot r^N \mod N^2$$

Déchiffrement. `Dec()`. Pour retrouver le texte clair m, on commence par remarquer que :

$$c \equiv r^N \mod N$$

On obtient ainsi :

$$r = c^{N^{-1} \bmod \varphi(N)} \mod N.$$

De plus,

$$\begin{aligned}
\frac{c}{r^N} &= (1+N)^m \mod N^2 \\
&= 1 + m \cdot N + N^2 \cdot \left(\sum_{i=2}^{m} \binom{m}{i} N^{i-2} \right) \mod N^2 \\
&= 1 + m \cdot N \mod N^2.
\end{aligned}$$

D'où :

$$m = \frac{(c \cdot r^{-N} \mod N^2) - 1}{N}.$$

On remarque que le calcul de r n'est possible qu'à l'aide de la clef privée $\mathsf{sk} = \varphi(N)$, qui reste secrète sous l'hypothèse de la difficulté de la factorisation.

Remarque : Le nombre r tiré aléatoirement doit également être différent à chaque chiffrement. Le cryptosystème de Pailler n'est considéré comme sûr aujourd'hui que pour des très grands nombres : n doit faire au moins 2048 bits.

3 Avant d'implémenter le cryptosystème de Pailler

Le but de ce devoir maison est donc d'implémenter le cryptosystème de Pailler. Avant de faire cela, il va s'agir de générer correctement les paramètres du cryptosystème.

3.1 Quelle langage ? Quelles fonctionnalités ?

Nous allons y aller pas par pas. Il faut que vous commenciez par choisir un langage de programmation. Une fois, ce langage choisi, comme nous allons travailler avec des nombres de très grande taille, bien plus grande que les longueurs standards utilisées (64 ou 128 bits), il faut trouver la bibliothèque de grands nombres adaptée au langage choisi. Par exemple, si vous décidez de programmer en C, la bibliothèque GMP[1] permet de travailler en multi-précision sur des nombres entiers de grande taille.

Question 1. Quel langage de programmation avez-vous choisi ? Quelle bibliothèque permettant de gérer des nombres entiers de grande taille allez-vous utiliser ? Quelles sont les opérations implémentées dans cette bibliothèque (multiplication, addition,...) ?

De plus, pour générer les valeurs (très grandes elles aussi), m, n et r, il faut être capable de générer des nombres aléatoires de très bonne qualité et dits cryptographiquement sûrs.

Question 2. En vous aidant d'internet, donnez la définition d'un nombre aléatoire cryptographiquement sûr. Selon le langage de programmation choisi, donnez le nom de la bibliothèque qui va vous permettre de générer ces nombres aléatoires.

Il est à noter que nous aurons besoin de nombres de au moins 1024 bits, il faut donc s'assurer que le générateur de nombres aléatoires utilisé produit une chaîne binaire aléatoire suffisament longue. Soit il le fait par défaut, soit vous avez besoin d'implémenter une fonction intermédiaire `BigRandom` qui prend en entrée plusieurs petits nombres aléatoires et produit en sortie un nombre aléatoire de la bonne taille.

Avant d'implémenter `KeyGen()`, il faut également trouver une bibliothèque capable de dire si un très grand nombre est premier ou non. Cela est indispensable.

Question 3. Quel test utilise-t'on habituellement pour savoir si un nombre est premier ou non ? Ce test est dit probabiliste, qu'est ce que cela signifie ? Selon le langage que vous avez choisi quelle librairie et quelle fonction permet d'appeler ce test ?

3.2 Implémentation de l'exponentiation binaire

Pour faire les différents calculs du cryptosytème de Pailler, il faut également être capable de calculer des exponentiations modulaires. La méthode la plus rapide pour faire cela est dite méthode d'exponentation binaire et elle est décrite à l'Annexe A. Il s'agit donc d'implémenter ici cette méthode sous la forme d'une fonction `ExpMod()` qui prend en entrée N, c et a et qui renvoie en sortie $A \equiv c^a \bmod N$.

Question 4. Implémentez la fonction `ExpMod()`. Testez-la sur 10 très grands nombres différents. Est ce que la librairie que vous avez choisi implémente cette méthode ? Par quelle fonction ? Si, oui, comparez vos résultats aux résultats obtenus avec la fonction de la librairie sur 10 très grands nombres.

3.3 Implémentation de `KeyGen()`

Question 5. Implémentez l'algorithme `KeyGen()` qui prend en entrée p et q, deux grands nombres premiers de 1024 bits chacun et renvoie en sortie $\mathsf{pk} = N$ et $\mathsf{sk} = \varphi(N)$.

[1] voir : `https://gmplib.org/`.

4 Implémenter le cryptosystème de Pailler

Normalement à cette étape vous avez tous les éléments pour implémenter le cryptosytème de Pailler décrit précédemment. Pour cela nous allons implémenter les deux fonctions décrites précédemment : `Enc()` et `Dec()` qui utiliseront les sorties de `KeyGen()`.

- `Enc()` est la fonction qui prend en entrée un message m de taille arbitraire et fournit le chiffré c de ce message à l'aide de la clé publique `pk` produite par la fonction `KeyGen()`.
- `Dec()` est la fonction qui déchiffre le message m à partir de c à l'aide de la clé secrète `sk` produite également par la fonction `KeyGen()`.

Question 6. Implémentez ces deux fonctions. Testez-la en vérifiant que pour 100 valeurs différentes de m (un grand message de taille arbitraire) vous obtenez bien des chiffrés c et surtout des déchiffrés égaux à m.

Ca y est, normalement, vous avez un cryptosystème de Pailler qui fonctionne !

5 Implémenter la propriété homomorphique du cryptosystème de Pailler

Nous allons à présent nous intéresser à vérifier une propriété particulière du cryptosystème de Pailler, à savoir que le produit de deux chiffrés, si on le déchiffre correctement, donne la somme des deux clairs.

Le cryptosystème de Paillier est un homomorphisme additif, c'est-à-dire qu'avec la clef publique, un chiffré $c_1 = \mathsf{Chiffrer}(m_1)$ et un chiffré $c_2 = \mathsf{Chiffrer}(m_2)$, il est possible de construire un chiffré $c_{1+2} = \mathsf{Chiffrer}(m_1 + m_2)$ sans connaître ni m_1 ni m_2.

Cette opération s'effectue en multipliant c_1 et c_2, ce qui mène à:

$$\begin{aligned} c_1 \cdot c_2 &= (1+N)^{m_1} r_1^N \cdot (1+N)^{m_2} \cdot r_2^N \bmod N^2 \\ &= (1+N)^{m_1+m_2} (r_1 \cdot r_2)^N \bmod N^2 \end{aligned}$$

Qui correspond à un chiffré de $m_1 + m_2 \bmod \varphi(N)$ sous l'aléa $r_1 \cdot r_2$.

- Grâce à la fonction `Enc()`, on chiffre deux messages m_1 et m_2 pour obtenir les couples de chiffrés correspondants c_1 et c_2. Puis on calcule $c_{1+2} \equiv c_1 \times c_2$.
- Grâce à la fonction `Decrypt()`, on déchiffre c_{1+2} sous l'aléa $r_1 \cdot r_2$ et on vérifie que le résultat est égal à $m_1 + m_2$.

Question 7. Vérifiez cette propriété en chiffrant deux messages différents m_1 et m_2 avec la fonction `Enc()`, puis en calculant $c_{1+2} = c_1 \times c_2$ et enfin en déchiffrant via la fonction `Dec()` le chiffré c_{1+2} pour récupérer m sous l'aléa $r_1 \cdot r_2$. Vérifier ensuite que $m = m_1 + m_2$. Répétez ce test 10 fois.

Ce type de propriété s'appelle une propriété homomorphique (c'est-à-dire qu'on est capable de faire des opérations sur les chiffrés). Elle peut être un avantage dans certains contextes comme le vote électronique où l'on cherche à connaître une somme de votes et non chaque vote individuellement.

A Calculer une exponentiation modulaire à l'aide de la méthode dite d'exponentiation binaire

Lorsque l'on cherche à calculer g^a, la méthode naïve consiste à multiplier g par lui-même a fois. Ce n'est pourtant pas la méthode la plus rapide sur les ordinateurs, la meilleure méthode consiste à décomposer a en écriture binaire $a = \sum_{i=0}^{d} a_i 2^i$ où d est la taille en bits de a et où $a_i \in \{0, 1\}$. On a alors : $g^a = g^{a_0}(g^2)^{a_1} \cdots (g^{2^d})^{a_d}$. Il faut ainsi d opérations pour calculer les g^{2^i} et d opérations pour former le produit des $(g^{2^i})^{a_i}$, soit, au total, $2 \cdot d$ opérations. Ainsi l'algorithme le plus rapide pour calculer une exponentiation est le suivant dans sa version récursive qui calcule g^a pour a strictement positif :

$$\texttt{puissance}(g, a) = \begin{cases} g, & \text{si } a{=}1 \\ \texttt{puissance}(g^2, a/2), & \text{si } a \text{ est pair} \\ g \times \texttt{puissance}(g^2, (a-1)/2), & \text{si } a > 2 \text{ est impair} \end{cases}$$

Cet algorithme correspond à une élévation à la puissance classique. Comme nous souhaitons ici travailler modulo p pour calculer $g^a \bmod p$, tous les calculs de l'algorithme récursif doivent être faits modulo p.

B Description du `main()` attendu et des sorties dans `test.txt`

Le `main()` attendu implémentera les 5 premières occurences des tests demandés dans les différentes questions et les résultats de ces tests seront affichés dans le fichier `test.txt`.